

Document number	P0504R0
Date	2016-11-09
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Revisiting in-place tag types for any/optional/variant

Introduction

[P0032R3](#) was accepted in Oulu, although the part of the proposal changing the `in_place` tag types was controversial and there were two polls taken, for the complete proposal and only the less controversial part of it. In the end the complete proposal was accepted, however there are NB ballot comments pointing out problems with the controversial part. This proposal undoes that part to address CH 3 (the first) in [P0488R0](#), and also resolves GB 46 in the process.

Rationale

The new tag types are function references, to one of a set of overloaded functions and function templates, in order to allow `in_place` to be an overloaded name that can be used with no template argument list, or a template argument list consisting of either a single type argument or a single non-type argument. This is considered to be too clever and liable to confuse users. More importantly, they introduce ambiguities and errors for cases that work fine with traditional structs as tag types.

When attempting to pass the `in_place` tag (without arguments) as a deduced argument for a function template the compiler doesn't know that the non-template function is intended, and so the name is ambiguous. This example fails to compile, because the second `in_place` is ambiguous:

```
optional<optional<int>> o(in_place, in_place, 1);
```

Additionally, when passing the tag types by value they decay from function references to function pointers, and so are no longer usable where the reference is needed as a tag.

Discussions on the reflectors and the CH 3 ballot comment suggest reverting to simple tag types using structs, which have no surprising behaviour. There have been no compelling arguments for keeping the function reference forms.

Proposed Wording

Originally `in_place_t` was defined in `<optional>` and `in_place_type_t` and `in_place_index_t` was defined in `<variant>`, but `in_place_index_t` is also needed in `<any>` as well. I propose putting all of them in `<utility>` (where P0032R3 defined them) but it would be possible to put them back in their original locations, and say that `<any>` also makes `in_place_type_t` and `in_place_type` visible (similar to what we do for the range access function in `[iterator.range]`).

[LWG 2744](#), currently in Tentatively Ready status, adds a SFINAE constraint to `any(ValueType&&)` related to the `in_place_type_t` tag. That constraint needs to be changed to check `decay_t<ValueType>` not just `ValueType`. A similar constraint is needed for the `variant(T&&)` constructor.

Edit the end of the synopsis in `[utility]`:

```
// 20.2.7, in-place construction
struct in_place_tag {
    in_place_tag() = delete;
};
using in_place_t = in_place_tag(&)(unspecified );
template <class T>
    using in_place_type_t = in_place_tag(&)(unspecified <T>);
template <size_t I>
    using in_place_index_t = in_place_tag(&)(unspecified <I>);
in_place_tag in_place(unspecified );
template <class T>
    in_place_tag in_place(unspecified <T>);
template <size_t I>
    in_place_tag in_place(unspecified <I>);
struct in_place_t {
    explicit in_place_t() = default;
};
inline constexpr in_place_t in_place{};
template <class T>
```

```

struct in_place_type_t {
    explicit in_place_type_t() = default;
};
template <class T>
    inline constexpr in_place_type_t<T> in_place_type{};
template <size_t I>
    struct in_place_index_t {
        explicit in_place_index_t() = default;
    };
template <size_t I>
    inline constexpr in_place_index_t<I> in_place_index{};

```

Strike subclause 20.2.7 [utility.inplace] entirely.

20.2.7.In-place construction [utility.inplace]

~~-1- The in_place_t, [...]~~

~~-2- Remarks: Calling [...]~~

Modify 20.8.3.1 [any.cons] as indicated, **after** applying the resolution of [LWG 2744](#):

```

template<class ValueType>
    any(ValueType&& value);

```

-6- Let T be equal to `decay_t<ValueType>`.

-7- *Requires:* T shall satisfy the `CopyConstructible` requirements. If `is_copy_constructible_v<T>` is `false`, the program is ill-formed.

-8- *Effects:* Constructs an object of type `any` that contains an object of type T direct-initialized with `std::forward<ValueType>(value)`.

-9- *Remarks:* This constructor shall not participate in overload resolution if `decay_t<ValueType>` is the same type as `any` or if `ValueType` is a specialization of `in_place_type_t`.

Modify 20.8.4 [any.nonmembers]:

```

template <class T, class ...Args>
    any make_any(Args&& ...args);

```

-2- *Effects:* Equivalent to: `return any(in_place_type<T>, std::forward<Args>(args)...);`

```

template <class T, class U, class ...Args>
    any make_any(initializer_list<U> il, Args&& ...args);

```

-3-

Effects: Equivalent to: `return any(in_place_type<T>, il, std::forward<Args>(args)...);`

Modify 20.7.2.1 [variant.ctor]:

```

template<class T> constexpr variant(T&& t) noexcept(see below );

```

-12- Let T_j be a type that is determined as follows: build an imaginary function $FUN(T_i)$ for each alternative type T_i . The overload $FUN(T_j)$ selected by overload resolution for the expression $FUN(std::forward<T>(t))$ defines the alternative T_j which is the type of the contained value after construction.

-13- *Effects:* Initializes `*this` to hold the alternative type T_j and direct-initializes the contained value as if direct-non-list-initializing it with `std::forward<T>(t)`.

-14- *Postconditions:* `holds_alternative<Tj>(*this)` is `true`.

-15- *Throws:* Any exception thrown by the initialization of the selected alternative T_j .

-16- *Remarks:* This function shall not participate in overload resolution unless `is_same_v<decay_t<T>, variant>` is `false`, unless `decay_t<T>` is neither a specialization of `in_place_type_t` nor a specialization of `in_place_index_t`, unless `is_constructible_v<Tj, T>` is `true`, and unless the expression $FUN(std::forward<T>(t))$ (with FUN being the above-mentioned set of imaginary functions) is well formed.

[Note:

```
variant<string, string> v("abc");
```

is ill-formed, as both alternative types have an equally viable constructor for the argument. — *end note*]

-18- The expression inside `noexcept` is equivalent to `is_nothrow_constructible_v<Tj , T>`. If *Tj*'s selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.

Acknowledgments

Thanks to The Elf for implementing and testing these changes in libstdc++ and reviewing this proposal. Thanks to Agustín Bergé for pointing out that the constructors need to be constrained.